



**PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL  
INTEGRAL  
FORMATO ENTREGA DE EVIDENCIAS**

**Taller investigativo: Funciones Síncronas y Asíncronas en  
JavaScript**

**Presentado a:** Instructora Alexandra Muñoz

**Por Aprendiziz:** **Nicolas Ruiz Colorado**

**Ficha:** 3312932

**Competencia:**

Tecnólogo en Análisis y Desarrollo de Software  
Servicio Nacional de Aprendizaje SENA  
Centro de Teleinformática y Producción Industrial  
Regional Cauca

Popayán, día **15** de **septiembre** del año **2026**



**PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL  
INTEGRAL  
FORMATO ENTREGA DE EVIDENCIAS**

## **Tabla de Contenido**

|                                                                             |    |
|-----------------------------------------------------------------------------|----|
| Funciones Síncronas y Asíncronas en JavaScript.....                         | 6  |
| 1. Que es una función Sincrónica .....                                      | 6  |
| 2. Que es una función Asíncrona.....                                        | 6  |
| 3. Como se ejecuta una función síncrona .....                               | 6  |
| 4. Como se ejecuta una función Asíncrona .....                              | 6  |
| 5. Que significa que una operación sea bloqueante.....                      | 6  |
| 6. Que significa que una operación sea no bloqueante.....                   | 7  |
| 7. Que es una promise.....                                                  | 7  |
| 8. Para que sirve async y await.....                                        | 7  |
| 9. Que problemas pueden solucionarse utilizando programación asíncrona..... | 7  |
| 10. Tabla comparativa .....                                                 | 7  |
| 11. Comprendiendo el funcionamiento .....                                   | 9  |
| 11.1. Que mensaje aparece primero.....                                      | 9  |
| 11.2. Que mensaje aparece segundo.....                                      | 9  |
| 11.3. Que mensaje aparece tercero .....                                     | 10 |



**PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL  
INTEGRAL  
FORMATO ENTREGA DE EVIDENCIAS**

|       |                                                                                                            |    |
|-------|------------------------------------------------------------------------------------------------------------|----|
| 11.4. | La función saludar() debe terminar para continuar con la siguiente instrucción                             | 10 |
| 11.5. | Este código es síncrono o asíncrono .....                                                                  | 10 |
| 11.6. | Explique por qué.....                                                                                      | 10 |
| 12.   | Analizando una función asíncrona .....                                                                     | 10 |
| 12.1. | Que mensaje aparece primero.....                                                                           | 11 |
| 12.2. | Que mensaje parece inmediatamente después .....                                                            | 11 |
| 12.3. | Que mensaje aparece aproximadamente después de 2 segundos .....                                            | 11 |
| 12.4. | Por que fin aparece antes de proceso terminado.....                                                        | 11 |
| 12.5. | El programa se detiene durante los 2 segundos .....                                                        | 11 |
| 12.6. | Que característica de la programación asíncrona se puede observar....                                      | 11 |
| 13.   | Prediga el resultado .....                                                                                 | 11 |
| 13.1. | Orden esperado .....                                                                                       | 12 |
| 13.2. | Por que B no aparece inmediatamente después de A, aunque el tiempo establecido en setTimeout() sea 0 ..... | 12 |
| 14.   | Funciones con async y await .....                                                                          | 12 |
| 14.1. | Que devuelve al obtenerUsuario() .....                                                                     | 13 |
| 14.2. | Que representa resolve() .....                                                                             | 14 |



**PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL  
INTEGRAL  
FORMATO ENTREGA DE EVIDENCIAS**

|       |                                                                                                                           |    |
|-------|---------------------------------------------------------------------------------------------------------------------------|----|
| 14.3. | Que función esta definica como async .....                                                                                | 14 |
| 14.4. | Que hace await.....                                                                                                       | 14 |
| 14.5. | Que ocurre mientras se esperan los 2 segundos .....                                                                       | 14 |
| 14.6. | Por que async y await facilitan la lectura del codigo asíncrono .....                                                     | 14 |
| 14.7. | Que ocurriría si se eliminara el await .....                                                                              | 14 |
| 15.   | Eleccion Sincrona o Asincrona.....                                                                                        | 15 |
| 16.   | Situación problema .....                                                                                                  | 15 |
| 16.1. | Utilizaria una función asíncrona o síncrona .....                                                                         | 16 |
| 16.2. | Por que.....                                                                                                              | 16 |
| 16.3. | Qué problema podría presentarse si la aplicación bloqueara<br>completamente la interfaz mientras espera la respuesta..... | 16 |
| 16.4. | Qué tecnologías de JavaScript podrían utilizarse .....                                                                    | 16 |
| 17.   | Reto Practico.....                                                                                                        | 16 |
| 18.   | Reflexión Final .....                                                                                                     | 18 |
| 18.1. | Cual es la principal diferencia entre una función sincrónica y una<br>asincrónica                                         | 18 |
| 18.2. | Por que las operaciones que dependen de un servidor normalmente se<br>manejan de forma asíncrona.....                     | 18 |



**PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL  
INTEGRAL  
FORMATO ENTREGA DE EVIDENCIAS**

- 18.3. Que ventaja ofrece `async/await` frente a trabajar directamente `then()`.. 18
- 18.4. Una función asíncrona significa necesariamente que se ejecuta en otro  
hilo 18
- 18.5. Mencione tres situaciones reales en las que utilizaría programación  
asíncrona 19



## **PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS**

### **Funciones Síncronas y Asíncronas en JavaScript**

#### **1. Que es una función Sincrónica**

Es una función que se ejecuta en una secuencia directa, es decir para que se pueda realizar una tarea se debe haber terminado por completo la tarea anterior.

#### **2. Que es una función Asíncrona**

Es una función en la cual se puede iniciar una tarea de larga duración y las cuales permiten que esta tarea se siga ejecutando y a su vez se puedan ejecutar otros procesos sin que la principal se detenga.

#### **3. Como se ejecuta una función síncrona**

Se coloca directamente en la pila de llamadas (call stack) del motor JavaScript y se ejecuta de manera secuencial hasta retornar el valor

#### **4. Como se ejecuta una función Asíncrona**

Se inicia el (call stack), pero las tareas pesadas o de espera, se delegan al entorno y cuando finalizan pasan a una cola de tareas y el bucle de eventos las reintroduce en el call stack cuando quede libre.

#### **5. Que significa que una operación sea bloqueante**

Significa que el flujo de trabajo o la interfaz de usuario se congela durante el tiempo que tarda en completarse impidiendo la ejecución del de cualquier otra línea de código



**PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL  
INTEGRAL  
FORMATO ENTREGA DE EVIDENCIAS**

**6. Que significa que una operación sea no bloqueante**

Significa que el programa solicita la operación y continua inmediatamente ejecutando las siguientes instrucciones sin congelar la interfaz.

**7. Que es una promise**

Es un evento que representa el resultado eventual de una operación asíncrona puede ser Pending(pendiente), fulfilled(exitosa) y rejected(rechazada con error)

**8. Para que sirve async y await**

async permite definir una función asíncrona la cual devuelve una promise

await permite esperar el resultado de una promise dentro de una función asíncrona

**9. Que problemas pueden solucionarse utilizando programación asíncrona**

Ayuda a que la interfaz no se quede congelada mientras se espera una respuesta en operaciones como manejo de APIs, lectura de archivos.

**10. Tabla comparativa**

| Característica   | Funciones Síncronas     | Funciones Asíncronas                                                    |
|------------------|-------------------------|-------------------------------------------------------------------------|
| Como se ejecutan | Operación por operación | Se inician operaciones y se esperan los resultados sin detener el flujo |



**PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL  
INTEGRAL  
FORMATO ENTREGA DE EVIDENCIAS**

|                       |                                                |                                                                     |
|-----------------------|------------------------------------------------|---------------------------------------------------------------------|
| Orden de ejecución    | Se respeta el orden del código                 | Se pueden realizar múltiples operaciones                            |
| Bloquean la ejecución | Si mientras se realiza una operación           | Permiten continuar con otras tareas mientras se espera un resultado |
| Esperan una operación | Si, antes de continuar la siguiente            | Puede esperar una respuesta sin bloquear todo el programa           |
| Tiempo de respuesta   | Depende de cuanto tarda cada operación         | Puede continuar con otras tareas durante la espera                  |
| Uso de promise        | No es necesario                                | Se utilizan con mucha frecuencia                                    |
| Uso de async/await    | No es necesario                                | Se utilizan para trabajar con promise de forma sencilla             |
| Ejemplos              | Sumar números o validar datos                  | Consultar una API                                                   |
| Cuando se utilizan    | Para operaciones rápidas o cálculos inmediatos | Para operaciones con dependencias externas o que sean tardadas      |



**PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL  
INTEGRAL  
FORMATO ENTREGA DE EVIDENCIAS**

|             |                                                                     |                                                                                                       |
|-------------|---------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| Ventajas    | Son fáciles de entender en el cual se sigue un orden lógico y claro | Se evitan esperas innecesarias y facilitan el manejo de las tareas que son lentas o de larga duración |
| Desventajas | Una operación lenta que puede detener el flujo                      | Requiere conocimiento de algunos parámetros como el promise y los errores de orden de ejecución       |

## **11. Comprendiendo el funcionamiento**

```
console.log("Inicio")
function saludar(){
  console.log("Hola aprendiz")
}
saludar()
console.log("Fin")
```

### **11.1. Que mensaje aparece primero**

Inicio

### **11.2. Que mensaje aparece segundo**

Hola aprendiz



## PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

### 11.3. *Que mensaje aparece tercero*

Fin

### 11.4. *La función saludar() debe terminar para continuar con la siguiente instrucción*

Si la función debe terminar para que aparezca el mensaje Fin

### 11.5. *Este código es síncrono o asíncrono*

Este código es síncrono

### 11.6. *Explique por qué*

Este código es síncrono porque tiene un orden lógico en el cual primero se ejecuta una operación, cuando esta finaliza se ejecuta la siguiente en este caso, primero se muestra inicio, después se ejecuta la función saludar(), la cual muestra Hola aprendiz y cuando la función termina se ejecuta la siguiente la cual es Fin, (también podremos observar la NO presencia de un Promise o el async o el await)

## 12. Analizando una función asíncrona

```
console.log("Inicio")
setTimeout(()=>{
  console.log("Proceso terminado")
}, 2000)
console.log("Fin")
```



## PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

### 12.1. *Que mensaje aparece primero*

Inicio

### 12.2. *Que mensaje parece inmediatamente después*

Fin

### 12.3. *Que mensaje aparece aproximadamente después de 2 segundos*

Proceso terminado

### 12.4. *Por que fin aparece antes de proceso terminado*

Porque la función `setTimeout()` programa la ejecución del callback después del tiempo indicado y como es una función asíncrona que no se bloquea se ejecutan las tareas siguientes

### 12.5. *El programa se detiene durante los 2 segundos*

No se detiene, puede continuar ejecutando otras tareas mientras espera que llegue el momento de ejecutar el call back

### 12.6. *Que característica de la programación asíncrona se puede observar*

Se observa una operación que puede esperar un tiempo sin bloquear la ejecución de las siguientes

## 13. Prediga el resultado

```
console.log("A")  
setTimeout(()=>{
```



**PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL  
INTEGRAL  
FORMATO ENTREGA DE EVIDENCIAS**

```
    console.log("B")
  }, 0)
  console.log("C")
  setTimeout(()=>{
    console.log("D")
  },1000)
  console.log("E")
```

**13.1. Orden esperado**

```
> console.log("A")
  setTimeout(()=>{
    console.log("B")
  }, 0)
  console.log("C")
  setTimeout(()=>{
    console.log("D")
  },1000)
  console.log("E")

A VM104:1
C VM104:5
E VM104:9
< undefined
B VM104:3
D VM104:7
```

**13.2. Por que B no aparece inmediatamente después de A, aunque el tiempo establecido en setTimeout() sea 0**

Porque el setTimeout() no ejecuta el callback de manera inmediata, este se programa para poder ejecutarse justo después de del código sincrónico

**14. Funciones con async y await**

```
function obtenerUsuario(){
```



**PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL  
INTEGRAL  
FORMATO ENTREGA DE EVIDENCIAS**

```
return new Promise((resolve)=>{
  setTimeout(()=>{
    resolve("Usuario encontrado")
  },2000)
})
}
async function mostrarUsuario(){
  console.log("Buscando usuario...")
  const resultado = await obtenerUsuario()
  console.log(resultado)
  console.log("Proceso terminado")
}
mostrarUsuario()
```

```
> function obtenerUsuario(){
  return new Promise((resolve)=>{
    setTimeout(()=>{
      resolve("Usuario encontrado")
    },2000)
  })
}
async function mostrarUsuario(){
  console.log("Buscando usuario...")
  const resultado = await obtenerUsuario()
  console.log(resultado)
  console.log("Proceso terminado")
}
mostrarUsuario()
Buscando usuario... VM112:9
< ▶ Promise {<pending>}
Usuario encontrado VM112:11
Proceso terminado VM112:12
>
```

**14.1. Que devuelve al obtenerUsuario()**

Devuelve un promise que esta pendiente pero al cabo de 2 segundos cambia a usuario encontrado



**PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL  
INTEGRAL  
FORMATO ENTREGA DE EVIDENCIAS**

**14.2.      *Que representa resolve()***

Indica que la promise termino y entrego el resultado de la operación

**14.3.      *Que función esta definica como async***

La función mostrarUsuario()

**14.4.      *Que hace await***

Espera el resultado de la promise dentro de la función asíncrona y el codigo posterior se ejecuta cuando la promise se cumpla

**14.5.      *Que ocurre mientras se esperan los 2 segundos***

La función mostrarUsuario(), queda esperando el resultado pero el programa puede continuar con otras tareas

**14.6.      *Por que async y await facilitan la lectura del codigo asíncrono***

Porque permiten escribir el codigo con una estructura parecida a la síncrona, sin tener que encadenar muchos callbacks o then

**14.7.      *Que ocurriría si se eliminara el await***

La variable resultado recibiría la promise no directamente el texto Usuario encontrado



## PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

### 15. Eleccion Sincrona o Asincrona

| Situación                                          | S /<br>A | ¿Por qué?                                                         |
|----------------------------------------------------|----------|-------------------------------------------------------------------|
| Sumar dos números                                  | S        | Es un calculo rápido que no necesita una respuesta externa        |
| Validar que un campo no esté vacío                 | S        | Se puede comprobar directamente el valor del campo                |
| Consultar información de una API                   | A        | Se debe esperar una respuesta del servidor                        |
| Consultar una base de datos                        | A        | La consulta puede tardar y depende de una fuente externa.         |
| Esperar una respuesta de un servidor               | A        | Se necesita manejar una operación que puede tardar                |
| Calcular el total de una factura                   | S        | Es una operación matemática que normalmente se realiza en el acto |
| Leer información de un archivo de manera asíncrona | A        | Se utiliza una operación de lectura que puede tardar              |
| Realizar una animación con retraso                 | A        | Se programa la ejecución despued de un tiempo                     |
| Procesar una operación que depende de una API      | A        | El procesamiento necesita esperar el resultado de la API          |
| Mostrar el resultado de una operación matemática   | S        | Si el calculo ya termino mostrar el resultado es inmediato        |

### 16. Situación problema

Una empresa necesita desarrollar una aplicación web para consultar información de sus clientes.

Cuando el usuario presiona el botón:

**“Consultar cliente”**

la aplicación debe:

1. Enviar una solicitud al servidor.
2. Esperar la respuesta.
3. Recibir los datos del cliente.
4. Mostrar la información en pantalla.



## PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

### **16.1.      *Utilizaría una función asíncrona o síncrona***

Utilizaría una función asíncrona

### **16.2.      *Por que***

Porque la aplicación necesita enviar una solicitud al servidor y esperar una respuesta. Durante ese tiempo, la interfaz no debería quedar bloqueada

### **16.3.      *Qué problema podría presentarse si la aplicación bloqueara completamente la interfaz mientras espera la respuesta***

La aplicación podría quedarse congelada. El usuario no podría interactuar con los botones, escribir información o realizar otras acciones hasta que el servidor respondiera

### **16.4.      *Qué tecnologías de JavaScript podrían utilizarse***

- Promise: Representa el resultado futuro de la solicitud al servidor
- Async: Define una función que trabaja con operaciones asíncronas y devuelve una Promise
- Await: Espera el resultado de una Promise dentro de una función async
- Fetch: Permite realizar solicitudes HTTP, por ejemplo, para consultar información de una API

## **17. Reto Practico**

```
function consultarUsuario() {  
  return new Promise((resolve) => {  
    setTimeout(() => {
```



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL  
INTEGRAL  
FORMATO ENTREGA DE EVIDENCIAS

```
        resolve("Usuario encontrado");
    }, 3000);
});
}

async function mostrarUsuario() {
    console.log("Consultando usuario...");

    const resultado = await consultarUsuario();

    console.log(resultado);
    console.log("Proceso terminado");
}

mostrarUsuario();
```

```
> function consultarUsuario() {
    return new Promise((resolve) => {
        setTimeout(() => {
            resolve("Usuario encontrado");
        }, 3000);
    });
}

async function mostrarUsuario() {
    console.log("Consultando usuario...");

    const resultado = await consultarUsuario();

    console.log(resultado);
    console.log("Proceso terminado");
}

mostrarUsuario();
Consultando usuario... VM154:10
< ▶ Promise {<pending>}
Usuario encontrado VM154:14
Proceso terminado VM154:15
> |
```



## PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

### 18. Reflexión Final

#### 18.1. *Cual es la principal diferencia entre una función sincrónica y una asincrónica*

La principal diferencia es que una función síncrona ejecuta sus instrucciones en orden y espera a que cada operación termine. Una función asíncrona permite manejar operaciones que tardan sin detener todo el flujo del programa.

#### 18.2. *Por que las operaciones que dependen de un servidor normalmente se manejan de forma asíncrona*

Principalmente por el tiempo de respuesta del servidor y así la aplicación pueda seguir funcionando mientras espera esa respuesta

#### 18.3. *Que ventaja ofrece async/await frente a trabajar directamente then()*

La ventaja es que el código resulta más fácil de leer y entender

#### 18.4. *Una función asíncrona significa necesariamente que se ejecuta en otro hilo*

No necesariamente una función asíncrona no significa automáticamente que se ejecute en otro hilo, muchas utilizan el ciclo de eventos para continuar el programa mientras esperan



**PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL  
INTEGRAL  
FORMATO ENTREGA DE EVIDENCIAS**

**18.5.      *Mencione tres situaciones reales en las que utilizaría programación  
asíncrona***

- Consultar datos de una API
- Guardar información en una base de datos y esperar la respuesta del servidor
- Descargar o leer un archivo sin bloquear la interfaz de la aplicación